Dataset A (classification): for the first set of 5 small programming tasks use the Oasis dataset provided by the *nilearn* package, which can be downloaded using the code below. This is a dataset of structural brain scans (aka voxel-based morphometry) for male and female individuals.

```
from nilearn import datasets

import numpy as np

from nilearn.input_data import NiftiLabelsMasker

brain_data = datasets.fetch_oasis_vbm(n_subjects=100)

yeo = datasets.fetch_atlas_yeo_2011()

masker = NiftiLabelsMasker(labels_img=yeo['thick_17'],
standardize=True, memory='nilearn_cache')

input_variables = masker.fit_transform(brain_data.gray_matter_maps)

output_variable = np.array(brain_data.ext_vars.mf == 'F',
dtype=int) #gives 1 for females and 0 for males
```

Note: it is expected that the models may not converge with the default max_iter, that is okay.

1. Use `sklearn.linear_model.LogisticRegression`(hyperparameter set to 1.0) **trained on all 17 input_variables** to perform uncertainty estimation. Draw **200 bootstrap samples** from the dataset, and for each of these bootstrap datasets, fit your model on all 100 subjects (you will train 200 models). Plot the distribution of model parameters `(.coef_)` using pyplot.boxplot(). The distribution of the model parameters should be on the y-axis and 17 input variables on the x axis; the mean of each box is the original point estimate of the model coefficient (i.e., the original dataset, not the bootstrap data).

2. Without predictive modeling; using 200 bootstrap iterations, **in each drawn bootstrap** dataset, subtract the *mean volume in an input variable across males* **from** the *mean volume across females*, for each **input variable**. Then, plot the 5% and 95% percentiles of the distribution of 200 bootstrapped differences for the purpose of hypothesis testing (dispersion on the y-axis and input variables on the x-axis). Hint: use np.percentile().

3. Define the training dataset as the **first 80 of the 100 subjects, then draw** 200 bootstrap samples of this training dataset. On each of the 200 bootstrap samples, train a logistic regression model `sklearn.linear_model.LogisticRegression`(hyperparameter set to 1.0) to distinguish male versus female participants. Then, building on those 200 fitted models, improve the prediction performance on the **remaining subjects as testing**

data (unseen 20% of the 100 subjects) by majority vote (i.e., select the most common class) across the 200 bootstrapped logistic regression models. Report this average classification performance (% number, no plot).

4. Define the **training dataset** as the same 80% of the 100 subjects as in Q3**,** draw **200 bootstrap samples** from this dataset, then train a logistic regression model `sklearn.linear_model.LogisticRegression` (hyperparameter set to 1.0) to distinguish male versus female participants. Average (np.mean) the 200 different sets of model parameters (.coef_) across the 200 fitted models to better identify the model parameters and plot them as bar plot (x-axis is the 17 input variable, y axis is the single mean-averaged parameter value across the bootstrap iterations). Use these averaged model parameters (coefficients) and the **remaining 20% of the 100 subjects as test data** in a single logistic regression model to *predict male versus female participants* using this new predictive model. Report the classification performance averaged across the 20 test subjects (% number, no plot).

5. Draw 200 bootstrap samples from **all 100 subjects** and fit a logistic regression model on each bootstrap sample. Then, perform cross-validation (**CV**) via **out-of-bag (OOB) evaluation** on the remaining (roughly ⅓) unseen (unsampled) subjects in each bootstrap iteration (recall lecture slides: only ~ 2/3 of the original dataset shows up in the bootstrap samples). Plot a scatterplot of the OOB CV accuracy for each bootstrap iteration (x-axis is the 200 iterations, y-axis is the OOB accuracy of the corresponding model).

Dataset B (regression): for the second set of 5 small programming tasks, use the Haxby dataset provided by the *nilearn* package, which can be downloaded using the code below. This is a dataset of functional brain scans (functional MRI measurements from one subject) acquired while individuals watched people's faces or house pictures.

```
from nilearn import datasets
import numpy as np
import pandas as pd
import nibabel as nib
from nilearn.image import new_img_like
from nilearn.plotting import plot_stat_map
from matplotlib import pylab as plt
haxby_dataset = datasets.fetch_haxby(subjects=1)
```

```
mask_filename = haxby_dataset.mask

mask_img = nib.load(mask_filename)

process_mask = mask_img.get_data()

picked_slice = 29

process_mask[..., (picked_slice + 1):] = 0

process_mask[..., :picked_slice] = 0

process_mask[:, 30:] = 0 # zero out anterior "half" of the brain

process_mask_img = new_img_like(mask_img, process_mask)

from nilearn.input_data import NiftiMasker

nifti_masker = NiftiMasker(smoothing_fwhm=8,
mask_img=process_mask_img)

func_filename = haxby_dataset.func[0]

fmri_masked = nifti_masker.fit_transform(func_filename)

labels = pd.read_csv(haxby_dataset.session_target[0], sep=" ")

conditions = labels['labels']

condition_mask = conditions.isin(['face', 'house'])

fmri_masked = fmri_masked[condition_mask]

output_variable = np.array(conditions[condition_mask] == 'face',
dtype=int)

print('The only relevant horizontal brain slice has %i voxels.' %
process_mask[..., picked_slice].sum())

#plot that slice to get an impression of where the input variables
are in the brain plot_stat_map(process_mask_img, cut_coords=[-9],
display_mode="z", cmap=plt.cm.RdBu_r)
```

Note: since there are many more input variables in this dataset, for all of the following questions only generate plots of the first 17 input variables (your models should still be trained using all input variables). For Q8 and Q9 use an 80/20% train/test split.

6. analogous to task 1 but using sklearn.linear_model.Ridge(default hyperparameters) instead of logistic regression.

7. analogous to task 2

8. analogous to task 3 but using sklearn.linear_model.Ridge(default hyperparameters). Instead of predicting the binary class and using majority vote to calculate classification accuracy, use the average of the 200 bootstrap predictions to score (Ridge.score()) the performance on the test subjects.

9. analogous to task 4 but using sklearn.linear_model.Ridge(default hyperparameters) and using the coefficient of determination instead of accuracy

10. analogous to task 5 but using sklearn.linear_model.Ridge(default hyperparameters) and using the coefficient of determination instead of accuracy